

Building AI Native Applications with Streamlit and Gradio

Topics

- e! Streamlit fundamentals: layout, widgets, session state, and caching
- e! Building a real-time streaming chat UI in Streamlit
- e! Gradio components: text, file, image inputs and markdown outputs
- e! Creating shareable Gradio demos and deploying to Hugging Face Spaces
- e! Wiring a Streamlit or Gradio front-end to a FastAPI backend
- e! Comparing Streamlit vs Gradio: when to use which

Learning Objectives

By the end of this lesson, you will be able to:

- e! Build AI applications using Streamlit layouts, widgets, state, and caching.
- e! Create real-time chat interfaces with streamed LLM responses.
- e! Design interactive Gradio interfaces with multiple input and output components.
- e! Connect Streamlit or Gradio front ends to FastAPI backends.
- e! Develop and deploy AI-powered web applications using Streamlit and Gradio.

Streamlit Fundamentals: Layout, Widgets, Session State, and Caching

Introduction to Streamlit

Streamlit is a pure-Python framework for building interactive web apps , no HTML, CSS or JavaScript required.



AI dashboards

Charts, metrics and data views.



Chat interfaces

Conversational front-ends
for LLMs.



Rapid prototyping

From script to shareable app fast.

It includes built-in widgets, charts, and layouts for creating data and AI applications.

Streamlit Layout and Widgets

Widget	What it does
<code>st.text_input()</code>	Collects a line of text from the user.
<code>st.button()</code>	Triggers an action when clicked.
<code>st.selectbox()</code>	Lets the user choose one option from a list.
<code>st.file_uploader()</code>	Uploads a file into the app.
<code>st.slider()</code>	Selects a numeric parameter within a range.

Managing State and Performance

```
state.py

import streamlit as st

if "messages" not in st.session_state:
    st.session_state.messages = []

st.session_state.messages.append(msg)
```

```
cache.py

import streamlit as st

@st.cache_data
def load_embeddings(path):
    return expensive_load(path) # Runs once
```

Session State:

`st.session_state` persists values across reruns, chat history, preferences and app state.

The Problem:

Without caching, **every rerun re-executes expensive operations**: reloading models or data on each interaction.

The Fix : Caching:

`@st.cache_data` caches data; `@st.cache_resource` caches models and connections, computed once, then reused.

Building a Real-time Streaming Chat UI in Streamlit

Building the chat interface and History

```
echo_bot.py

import streamlit as st

if "messages" not in st.session_state:
    st.session_state.messages = []

for m in st.session_state.messages:
    with st.chat_message(m["role"]):
        st.markdown(m["content"])

if prompt := st.chat_input("Say something"):
    st.session_state.messages.append(
        {"role": "user", "content": prompt}
    )

    with st.chat_message("assistant"):
        st.markdown(f"Echo: {prompt}")
```

Echo Bot : Running

What can you do?

Echo: What can you do?

Repeat after me

Echo: Repeat after me

Say something...

Connecting the LLM

```
llm_call.py

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {
            "role": "user",
            "content": prompt,
        }
    ],
)
st.markdown(response.choices[0].message.content)
```

Supported Providers:



Send prompt



Receive response



Display in Streamlit

Streaming Responses

```
stream.py

placeholder = st.empty()

full_response = ""

for token in llm.stream(prompt):
    full_response += token
    placeholder.markdown(full_response)
```

Accumulate, Don't Append Lines

Add each token to `full_response` and re-render the same `st.empty()` placeholder.

Shortcut

`st.write_stream(llm.stream(prompt))` handles the accumulation loop for you.

What the User Sees

An AI agent plans and acts towards goal...

Demonstration 1 : Streaming AI Responses Live in a Streamlit UI

Gradio Components: Text, File, Image Inputs and Markdown Outputs

Introduction to Gradio Components

Gradio components are reusable UI building blocks you wrap around a Python function, no frontend code required.



Input

Collect text, files and images from the user.



Output

Render the function's result back to the user.



Layout

Arrange components into rows, columns and tabs.

Gradio's modular components make it easy to design, test, and share interactive machine learning demos.

Input Components

Component	Use
 <code>gr.Textbox()</code>	Text input: prompts and questions.
 <code>gr.File()</code>	File upload: documents and datasets.
 <code>gr.Image()</code>	Image input: for vision models.

T

Type something...



Upload your file

Drag and drop or click to browse



File Components

Supports:

PDF



DOCX



CSV



TXT



Applications:



Document QA: Ask questions of a file's contents.



Resume analysis: Screen and summarize candidates.



Report processing: Extract insights from reports.



RAG pipelines: Feed documents into retrieval.

Output Components

Component	Renders
<code>gr.Markdown()</code>	Formatted text: headings, lists and emphasis.
<code>gr.Textbox()</code>	Plain text output.
<code>gr.Image()</code>	Images returned by the function.

Heading Title

- Item one
- Item two
- Item three

This is a plain text output.



Component Interaction

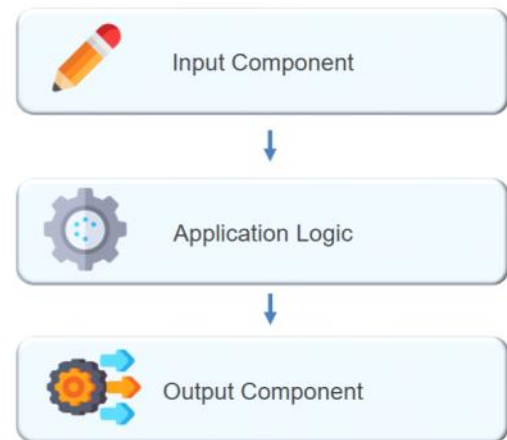
```
app.py

import gradio as gr

def predict(text):
    return f"**You said:** {text}"

demo = gr.Interface(
    fn=predict,
    inputs=gr.Textbox(),
    outputs=gr.Markdown(),
)

demo.launch()
```



Multi-Component Interfaces

```
app.py

import gradio as gr

def analyze(text, file, image):
    return run_ai(text, file, image)

demo = gr.Interface(
    fn=analyze,
    inputs=[
        gr.Textbox(),
        gr.File(),
        gr.Image(),
    ],
    outputs=gr.Markdown(),
)
```

One Function, Three Inputs



`gr.Textbox()` : prompt



`gr.File()` : document



`gr.Image()` : picture



`gr.Markdown()` response

Creating Shareable Gradio Demos and Deploying to Hugging Face Spaces

Sharing & Deploying Gradio Apps



Local launch

```
demo.launch()
```

Runs on your local machine only.



Temporary share

```
demo.launch(share=True)
```

A temporary public .gradio.live URL.



Hosted deployment

e.g. Hugging Face Spaces

A permanent public URL.

Advantages:



No frontend dev



Easy demos



Remote access



Rapid prototyping

Hugging Face Spaces

Hugging Face Spaces is a platform for hosting ML and AI demos that gives you a public URL with **no server management required.**



Supports Gradio and Streamlit applications



Provides a public URL for sharing demos



Hosts app files, dependencies, and runtime configuration



Enables quick deployment without managing servers



Gradio Deployment Workflow

- 1 Build and test the app locally
- 2 Create a Hugging Face Space & select Gradio as the SDK
The SDK is chosen at Space creation : before any files are added.
- 3 Add `app.py` and `requirements.txt`
- 4 Push the files to the Space repository
- 5 Wait for the Space to build
- 6 Share the public URL



Managing Dependencies for Deployment

List all libraries the app imports in `requirements.txt`

Include Gradio, LLM SDKs and processing libraries

Avoid unnecessary packages

Pin versions when stability matters

Test installation locally first

requirements.txt

```
gradio==4.44.0
openai==1.51.0
pypdf==5.0.1
```

Wiring a Streamlit or Gradio front-end to a FastAPI backend

Connecting Front-End Interfaces to FastAPI



Streamlit / Gradio

Provide the user interface



HTTP Requests



FastAPI backend

Logic · model calls · document processing · API workflows

Why a Separate Backend Matters?



Separation

UI stays independent of processing.



Reusable APIs

One API, many callers.



Complex Logic

Validation, uploads and async.



Multi-front end

Serve web, mobile and more.



Ready To Scale

Grow the backend on its own.

Communication Through HTTP Requests

FRONT-END ACTION

- ◆ User enters a prompt
- ◆ Uploads a file
- ◆ Clicks submit
- ◆ Backend calls the LLM
- ◆ Response returns to the UI



BACKEND ROLE

- ◆ Receives the input
- ◆ Validates and accepts the file
- ◆ Triggers processing
- ◆ Generates or retrieves the response
- ◆ Returns the result to be displayed

Demonstration 2 : Building and Deploying a Gradio AI Assistant with a FastAPI Backend

Comparing Streamlit vs. Gradio: When to Use Which

When to Use Streamlit?

Use Streamlit when the application richer layout, dashboards, data views, or multi-step interaction.



AI dashboards with charts and metrics

Multi-page or workflow-based AI apps

Internal business tools

Applications with filters, sidebars, and tables



When to Use Gradio?

Use Gradio when the goal is to quickly demonstrate an AI model, pipeline, or prototype with minimal interface setup.

- LLM chat demos
- Document question-answering demos
- Image or audio model demos
- Hugging Face Spaces deployments
- Public portfolio projects



Choosing the Right Front-End Tool

The choice should depend on the user experience, sharing needs, and application complexity.



Gradio



Quick public AI demo



Gradio



Hugging Face Spaces demo



Streamlit



Multi-step analytics app



Streamlit



Dashboard-style app

Summary

In this lesson, you have learned to:

- e! Build AI applications with Streamlit layouts, widgets, state, and caching.
- e! Create interactive Gradio interfaces with text, file, image, and Markdown components.
- e! Share Gradio applications using temporary public links.
- e! Deploy AI applications on Hugging Face Spaces.
- e! Connect Streamlit or Gradio to FastAPI backends.